RflySim / Real4Real

Preparation and blockers · 8 October 2026

Candidate: dynamics/control host and native visual baseline

A, B and coupled capture pending · C blocked

Real4Real inputs and target

Real images + calibrated poses → reconstruction → scene representation → new trajectories → observations → downstream evaluation

Residential, suburban and campus environments below 100 m; horizontal and vertical camera motion

Missing: imagery, calibration, reconstruction, split, held-out views, downstream model and approved fidelity thresholds

Component ownership

Python setpoints → PX4 SITL → actuator commands → CopterSim dynamics and simulated sensors → PX4 estimates

CopterSim truth → UE4 RflySim3D → mounted camera observations; UE supplies scene collisions and visual depth

MATLAB/Simulink authors models. Standalone installer may run the precompiled stock model without model compilation

Release and host gate

Official page names RflySimToolchain-v5.01-20260831.iso. Download link requires contact details; ISO URL, bytes, hash and EULA are missing

CopterSim lists Linux support. A complete licensed Free Linux visual and camera release has not been established

Candidate supported path: Windows native graphics + Python + CopterSim; PX4 in default WSL1 RflySim-20.04. Desktop, privilege and reboot access unverified

Resources and access

Provisional RTX A5000 ×1, 24 GB: expected sufficiency for one drone and one RGB camera is an estimate

Planning targets: recent 8-core CPU, 32 GB RAM, 500 GB disk / 300 GB free. Actual allocation, driver, OS and total price unknown

Supplied SSH candidate parsed; key already protected. No connection made. SSH access does not establish a GPU-backed Windows desktop

Frozen B route and criteria

NED metres: origin → (0,0,−2) → (2,0,−2) → (2,2,−2) → (0,0,−2) → land

Position error ≤0.25 m for a continuous 2 s dwell at every leg. Target setpoint rate 10 Hz; 75 s flight and separate 15 s cleanup watchdog

Minimum integration: takeoff, two waypoint transitions, landing/disarming. Full completion: every prescribed leg meets tolerance/dwell and confirms landing/disarming

A camera calibration and replay

Seven desired camera poses: baseline at (0,0,−2); +1 m north; +1 m east; 1 m higher; yaw +15°; pitch +10°; baseline again

640×480 RGB, requested 10 Hz, 21 s sequence replayed twice; 5 s warmup. Short rate test: 30 s after 5 s warmup

Repeatability: 0.02 m and 1°. Known-geometry reprojection: 2% of image dimension. Clearance, geometry, actual pose and clock mapping still block execution freeze

Flight-to-observation evidence

Commands → PX4 → dynamics → achieved simulator state → actual camera pose → timestamped RGB

B initial run without capture; reset and repeat the same route once with mounted capture if supported. Keep image/GPU/file workers separate from control

Record source clocks and receiver clocks, pose brackets, offsets, uncertainty, stale frames, gaps and missed deadlines. Reference poses and PX4 estimates cannot become camera truth

Runner findings

Recovered runner preserves evidence but cannot enforce deadlines independently of blocking SDK calls

PNG saving and GPU queries share the control loop; cached frames may be stale. No waypoint dwell or confirmed touchdown/disarm pass is implemented

Current entry point validates preparation and verifies hashes; simulator execution is disabled. Installed adapter, watchdog and capture isolation remain pending

Capture import and fidelity

Direct real images + poses: no inspected reconstruction loader established

Mesh route is documented through UE import, baking/cooking and XML MeshPath indexing. It is not a generic runtime OBJ/PLY loader; Free eligibility remains unresolved

Gaussian loader: no supported path established. Sparse capture coverage, altitude extrapolation and held-out fidelity remain blocked by missing calibrated assets

Gaussian renderer boundary

Pose replay exports a route. Native visual replacement changes the renderer. Closed-loop visual control also requires a compatible observation/action consumer

A separate renderer using CopterSim truth and calibrated extrinsics is inferred feasible; native image injection, reset/step and clock ownership are unresolved

UE collision and depth services must be retained or replaced with independently validated geometry. Implementation is outside preparation

Policy and task status

PX4 plus scripted waypoints is a controller, not a released autonomy policy

RflyArena publishes state-feedback RL weights: 25-dimensional state/error/reference input and throttle/body-rate output. Default-model and task compatibility are not established

No compatible R4R visual policy, scene/task or adapter supplied. C remains blocked; no unrelated weights or training

Applicability and decision

Indoor: candidate control/sensor integration. Outdoor residential/campus: candidate mesh baseline; reconstruction and novel-view accuracy unresolved

Sparse physical content and sparse captures need different tests. Racing needs gate perception, high-speed dynamics and collision timing evidence

Defer renderer adoption. Retain RflySim as a candidate dynamics/control host and native visual baseline, subject to host/release/SDK gates

Learning and preservation

Read SITLRun.bat, PX4MavCtrlV4.py, VisionCaptureApi.py, UE4CtrlAPI.py, model/scene XML and PX4 parameters: trace process ownership, NED/FRD, units, timestamps and modes

One writer on audit/rflysim; small evidence/deck in Git, bulk artifacts retained separately. Bundle relay can replace remote GitHub authentication

2 h active setup/debug and at most 2 repairs. Test transfer first; checkpoint during work/failure, finalize hashes and verify local media. Never terminate the instance automatically